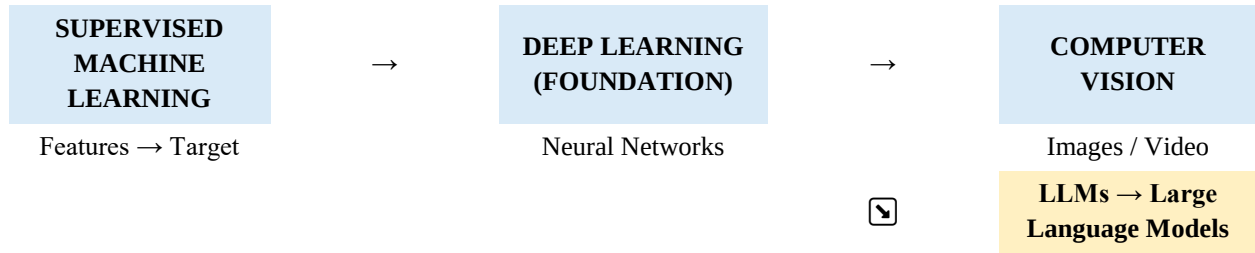


AI JOURNEY: SUPERVISED ML → DEEP LEARNING → CV & LLMs

Deep Learning is the bridge — it powers both Computer Vision and Large Language Models.



TWO LEADING LLMs — MODELS TO KNOW

ChatGPT	OpenAI Models	Claude	Anthropic Models
ChatGPT	GPT-5.5 — Flagship	Claude	Opus 4.8 — Most capable
	GPT-5.4 — Previous flagship		Sonnet 4.6 — Balanced
	GPT-5.4 mini — Affordable & fast		Haiku 4.5 — Fast & cheap

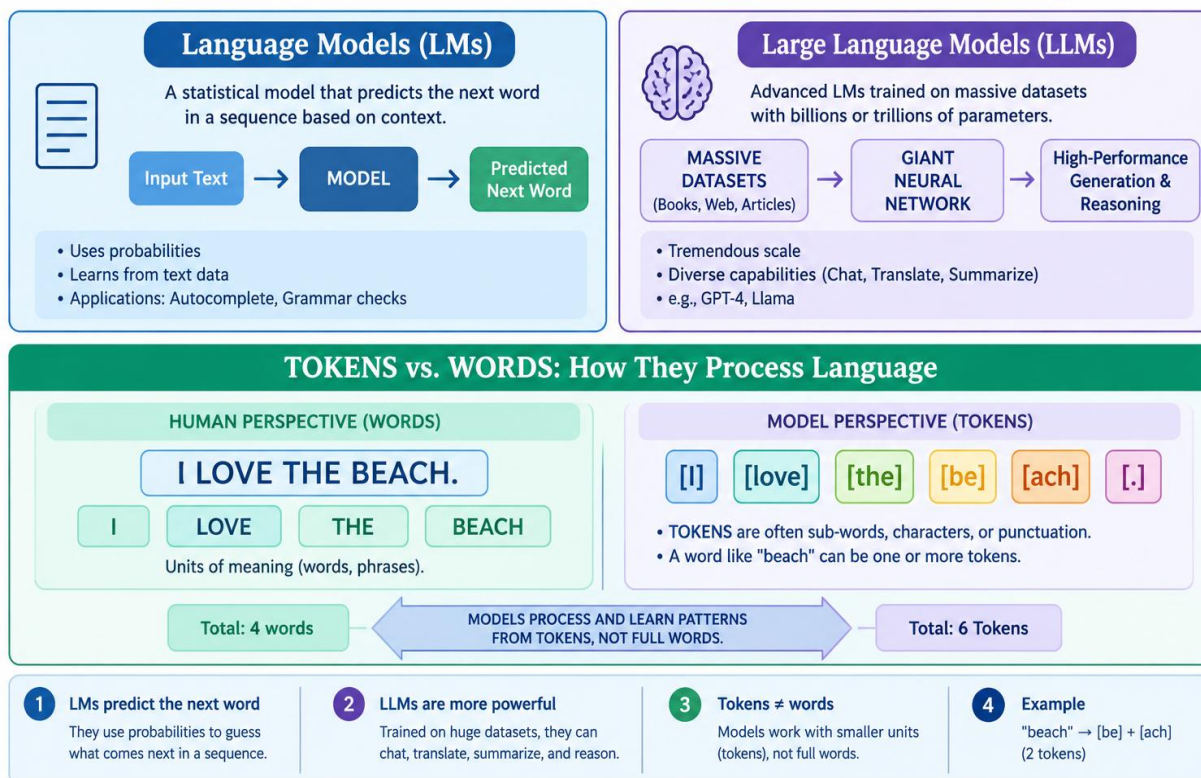
Bugungi kunda 2 ta yetakchilik qilayotgan LLM kompaniyalar bular yuqorida keltirilgan OpenAI va Anthropic kompaniyalaridir.

Model va kompaniya farqini anglab olish muhim basilar software va tool larni ham chalkashtirib yuboradi. Kompaniya bu modelni yaratgan kompaniya misol uchun OpenAI (GPT 3)ni Anthropic esa (Claude)ni yaratgan kompaniyalar. GPT-5.5, GPT-5.4, GPT-5.4 mini, Claude, GeminiAI bular hammasi model. Mana shu modellarni o'rab turuvchi app esa software, tool esa o'sha qurilgan modeldagi misol uchun search tugmasi, code yozadiganda run qiladigan tugmasi misol bo'la oladi.

LLM

Language model (LM)-(predict next word) bu konteksga asoslangan xolda ketma ket keyin keladigan so'z yoki gapni predict qiladi. So'zlarni qo'yishda ehtimollikdan foydalanib text datadan o'rganib predict qiladi. Asosan Autocomplete yoki grammar checks ni qiladi. Large language model (LLM)-bu esa LM ning advanced darajasi bo'lib juda katta dataset bilan milliard yoki trillionlab parameter tokens bilan train qiladi. Dataset ga misol qilib Books, Articles, Large texts. LLM bilan murakkabroq vazifalarni bajara olamiz. Chat qila olamiz, translate, large text → summarize kabilar misol bo'la oladi. Token vs Word: Token va so'zlar doim bir biriga teng emas yani bitta so'z doimo bitta token emas. Teng xolatlar ham bor. Model so'zlardan emas tokens dan o'rganadi.

Understanding Language Models



AI tadqiqotidagi inqiroz: pastdagi jadvalni batafsilroq tushuntirsam Avval mutaxassislar modellarni o'zlari qurgan va bu orqali uni nazariyasini to'liq tushunib nima qilayotganini bilgan . faqat software emas hardware qismidan ham xabardor bo'lishi kerak bo'lgan chunki ularni imkoniyatlarini bilib shunga qarab model qiulgan hozir esa Kodni ham LLM yozib beryabdi unchalik tushunmayabdi tushu8nsa ham Claude chatGPT qanday ishlashini bilmaydi qandayki asosiy parametrlari qanday qaysi usul qanday sharoitda qanday data bilan yaxshiligini bilmaydi sababi eng oxirgi GPT5.5 GPT 6 shularni qandfay ishlash jarayoni yoritilmagan hozircha secret.



AI Tadqiqotidagi Inqiroz

Zamonaviy AI tadqiqotchilari texnologiyadan tobora uzoqlashmoqda.



	10 yil oldin	Bugun
Modelni o'zi yaratardi	Modelni o'rganish, sozlash va qurish asosan inson tomonidan bajarilardi.	GPT-4, API-ni 1 qatorda chaqiradi.
Modelni o'zi o'qitardi	Ma'lumotlar to'planib, modelni o'qitish, parametrlarni sozlash, ko'p vaqt va resurs kerak edi.	Faqat prompt engineering bilan natija olish mumkin.
Dataset-ni o'zi to'plar edi	Ma'lumotlar qo'lda yig'ilar, tozalanar, formatlanar edi.	Tayyor datasetni yuklab oladi (yoki internetdan topadi).
Apparat chegaralarini tushunar edi	GPU, server, xotira kabi qurilmalar bilan ishlashni bilish kerak edi.	Hardware abstraklashgan (cloud va API orqali).
Ko'pchilik modellarga "qora quti" sifatida qarashadi	Model ichki ishlashini tahlil qilish, tushunish mumkin edi (oddiyroq).	Katta LLMlar ko'pchilik uchun "qora quti" — qanday ishlashini to'liq tushunish qiyin.



"Model o'zi train qiladi" nima degani?

Bu shuni anglatadiki, endi modelni yaratish va o'qitish jarayoni deyarli avtomatlashtirilgan. Siz faqat:

- 1 **Tayyor modelni** (masalan, GPT-4) API orqali chaqirasiz.
- 2 **Prompt** (savol yoki topshiriq) yozasiz.
- 3 **Model o'zi** sizga javob beradi.

Ya'ni, model ichki murakkab jarayonlarni (o'qitish, sozlash, optimallashtirish) o'zi bajaradi. Siz esa faqat natijani olasiz.



Asosiy farq

	Avval	Hozir
Modelni yaratish	Modeli ygratish	Model tayyor
O'qitish	Avtomatik (ichki)	Avtomatik (ichki)
Ma'lumot yig'ish	Tayyor dataset	Tayyor dataset
Hardware	Qo'lda boshqarish	Abstrakt (cloud/API)
Model tushunilishi	Osonroq	"Qora quti"



Xulosa

Hozirgi kunda AI texnologiyalari juda oddiy va qulay: siz kod yozmasdan, faqat **prompt** bilan kuchli natijalar olishingiz mumkin. Shu sababli, ko'pchilik endi texnik jarayonlarga emas, faqat natijaga e'tibor qaratmoqda.

Biroq, bu modelning ichki ishlashini tushunish va nazorat qilishni yanada muhim qiladi.

Frontier Modellar: Narxi va Muammo

Frontier modelar — eng kuchli, eng katta hisoblash resurslari va eng yuqori imkoniyatlarga ega sun'iy intellekt modellaridir.

1. Frontier Model nima?

Frontier model — bu hozirgi eng so'nggi avlod, eng katta va eng murakkab sun'iy intellekt modeli. Ular ko'p sonli parametrlar, katta ma'lumotlar va yuqori hisoblash quvvatlari bilan o'qitiladi.

Misol: GPT-4 (taxminan 1.8 trillion parametr, \$100M+ o'qitish xarajati).

2. Yechim (Purpose)

Kurs kichik ma'lumotlar (millionlab parametrlar) bilan ishlaydigan LLM qurish va o'qitishni o'rgatadi. Asosiy ko'nikma — **kichik resurslarda ham yaxshi natijaga erishadigan model yaratish**.

3. Qiyosiy misol (Frontier vs Kichik modellar)

Frontier model — shahar qurmoqchi bo'lgan muhandisga o'xshaydi. U juda katta resurslar, jamoa va vaqt bilan katta loyihani amalga oshiradi.
Kichik modellar esa — kichik arxitektura bilan, lekin aniq va maqsadli vazifalar uchun ishlaydi.

4. Asosiy xulosa

Frontier modellar — eng kuchli va eng qimmat, lekin har doim ham eng kichik vazifalar uchun kerak emas. Tadqiqot va yirik muammolar uchun ular muhimdir, lekin ko'pincha kichikroq va samarali modellar ham yetarli bo'ladi.

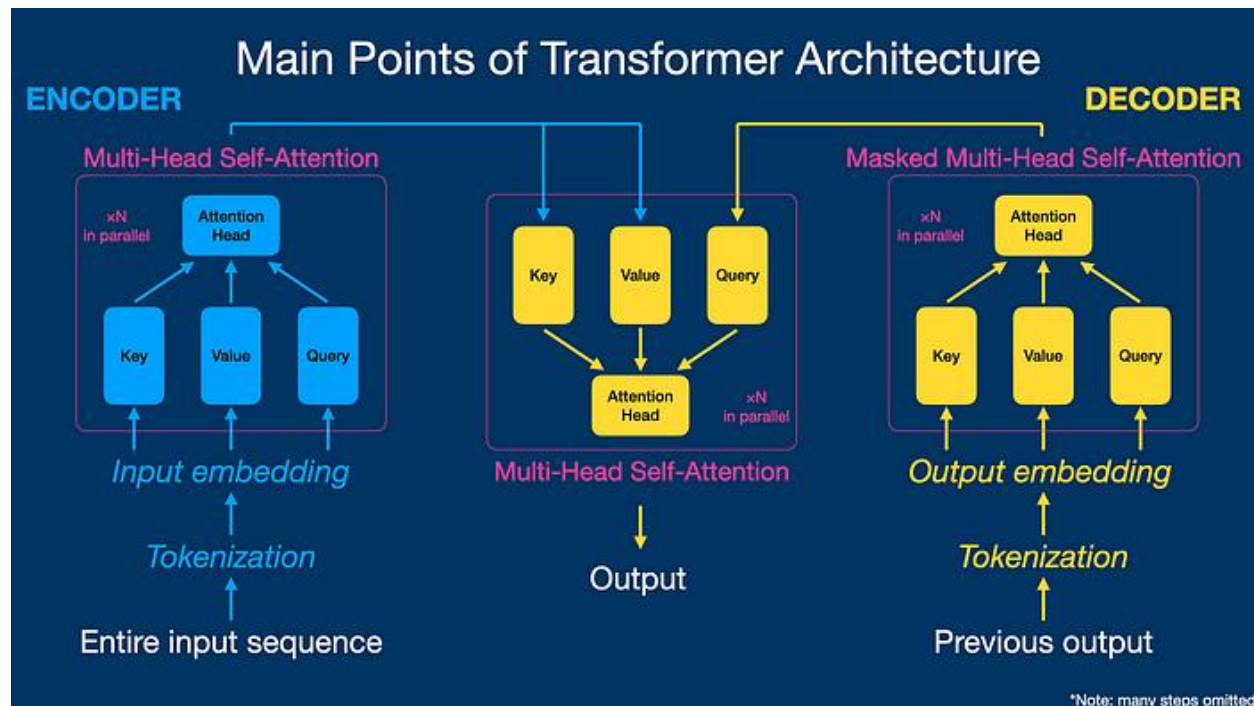
Frontier Modellar: Misollar va Narxlar

Model	Tizim / Ko'rsatkich	Qiymat (taxminiy)	Izoh
GPT-4 (OpenAI)	(OpenAI)	~1.8 trillion parametr	- Eng katta va kuchli modellardan biri - Matn, kod, rasm, tahlil va boshqalar - O'qitish narxi: \$100M+
xAI Colossus cluster	(xAI)	200,000 H100 GPU	- Juda katta hisoblash klasteri - Model o'qitish uchun - H100 GPU lar (NVIDIA)
H100 GPU narxi	(NVIDIA)	~\$30,000 (bitta)	- Bitta H100 GPU narxi - Juda yuqori hisoblash quvvati - Frontier modelar uchun ishlatiladi
xAI cluster narxi	(xAI)	~\$6 milliard (hardware)	200,000 ta H100 GPU klasteri - Infratuzilma va uskunalar narxi - Faqat apparat narxi (o'qitish narxi emas)
GPT-4 o'qitish narxi	(OpenAI)	\$100M - \$1B+	- Ma'lumotlar, hisoblash, jamoa, vaqt - Modelni o'qitish va sozlash xarajati - Aniq summa ochiq emas, taxminiy.

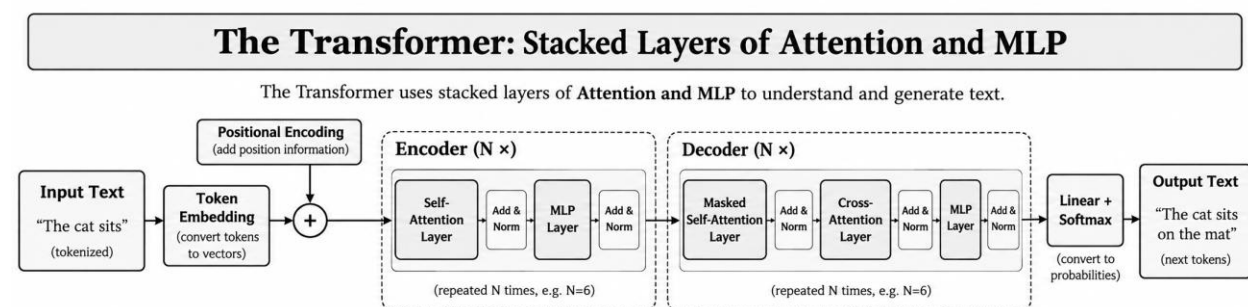
Atamalar va tushunchalar

Atama	Tushuntirish
Frontier model	Eng katta, eng kuchli va eng muakkam sun'iy intellekt modeli. Odatda milliardlab yoki trillionlab parametrlar bilan o'qitiladi.
Parametr	Model ichidagi o'rganiladigan o'zgaruvchi. Masalan, GPT-4 ~1.8 trillion parametrga ega.
O'qitish (Training)	Modelni ma'lumotlar asosida o'rganish jarayoni. Bu jarayon katta hisoblash quvvatini va vaqtni talab qiladi.
GPU	Parallel hisoblash uchun mo'ljallangan kuchli chip. Frontier modellarni o'qitishda ishlatiladi.
H100 GPU	NVIDIA'ning eng yangi va kuchli GPU laridan biri. AI modelini o'qitishda keng qo'llaniladi.
Xarajat	Modelni o'qitish va infratuzilma uchun sarflanadigan mablag'. Frontier modellarda bu juda katta bo'ladi.

Frontier models bu Hozirgi kundagi eng oxirgi, eng kuchli, avvalgiga nisbatan mukammalroq va ulkan compute harajat qila oladigan clusters larda train qilingan modellar hozirgi kunda misol OpenAI, Anthropic, Google shular yaratgan modellar. Biz frontier model qura olmaymiz misol uchun: bitta shahar qurmaymiz lekin aniq reja qilamiz, arxitekturasini qila olamiz uni maketini quramiz va keyin buni agar compute bo'lsa jamoa bo'lsa real frontier modelga ko'chiramiz.



Transformer bu LLM ning NN hisoblanadi va Transformer 2 ta asosiy layerdan 1. Attention layer 2. MLP layer pastda hozir rasmda yanada tushunarliroq qilib ko'rsataman.



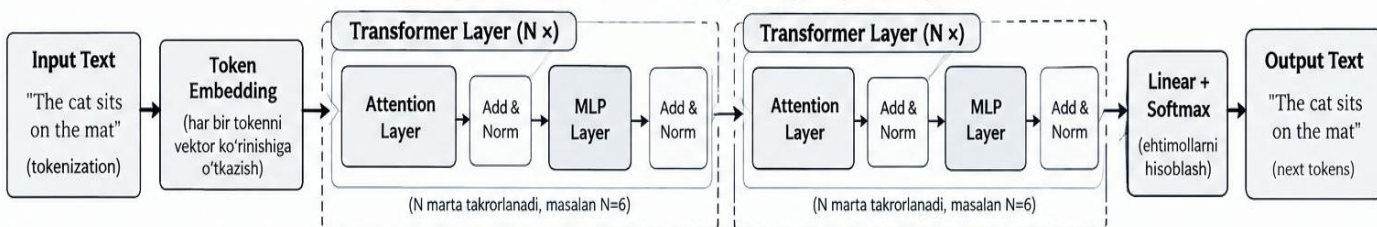
Bu yerga demak text kiritilyabdi "The cat sits ..." Self-attention va MLP layer birgalikda foydalanilib transformer yani LLM ning NN ki orqali output chiqayabdi yani predicting next tokens. Attention yoki MLP lardan alohida foydalansa bo'ladi lekin birgalikda yaxshiroq LLM quradi.

Kichik model vs Katta model:

Flop tushunchasi birinchi bilish kerak bo'lgan muhim tushuncha. FLOP — kompyuter modelni hisoblash paytida bajarayotgan bitta matematik amal. Misol uchun 2.3×4.7 bu bitta FLOP. FLOP va FLOPs farqli narsalar. FLOP bu bajarilgan Floating-Point Operations jami soni. FLOPs esa bir sekundda bajariladigan FLOP soni. Agar 1 FLOP = 1 ta matematik operatsiya deb olsak 1 sekundda 1FLOP bajarilsa bu 1FLOPs ga teng. Yoki 1 sekundda 1MFLOPs bajarilsa bu 1millionta matematik amal bajarildi degani. FLOP = model hisoblash vaqtida nechta matematik amal bajarilganini o'lchash birligi. FLOPs = bu amallarni qanchalik tez bajarayotganini o'lchaydi. Pastdagi rasmda aniqroq tushunib olamiz.

Kichik Model Katta Model Kabi Harakat Qilmaydi

Bu rasmda FLOP (Floating Point Operation) bo'yicha kichik va katta modelning farqi, shuningdek, attention va MLP bloklarining ishlash prinsipi ko'rsatilgan.



1 Model hajmi

Model hajmi — bu modelda nechta parametr (og'irlik) mavjudligini bildiradi. Parametrlar ko'p bo'lsa, model ko'proq ma'lumotni o'rganishi va murakkabroq munosabatlarni tushunishi mumkin.

Model	Parametrlar soni (taxminiy)
Kichik model (100M)	~100 million (1e8)
Katta model (100B+)	~100 milliard+ (1e11+)

Katta model ko'proq imkoniyatlarga ega, lekin ko'proq hisoblash resursi va xotira talab qiladi.

2 Attention vs MLP FLOPs

FLOPs — bu modelning bitta qatlamida bajariladigan matematik amallar soni (floating point operation).

Attention FLOPs:

- Asosan matritsa ko'paytmalari va softmax hisoblashlaridan iborat.
- Tokenlar o'rtasidagi aloqani (qaysi so'z qaysi so'zga e'tibor beradi) aniqlaydi.
- Ma'lumot hajmi oshgani sari FLOPs tez o'sadi (chunki kvadratga bog'liq).

MLP FLOPs:

- Har bir token uchun alohida fully connected (chiziqli) qatlamlar hisoblanadi.
- Attention ga qaraganda odatda kamroq FLOPs talab qiladi.

3 Xulosa

Attention optimizatsiyasi

Modelning e'tiborini (attention) qismlarga yo'naltirish usullarini yaxshilash orqali samaradorlik oshiriladi. Masalan: sparse attention, flash attention va h.k.

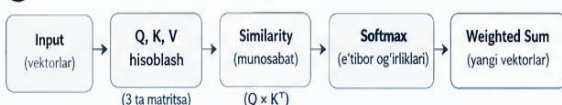
Attention optimizatsiyasi ahamiyati

Ma'lumotni yanada samarali qayta ishlash, ko'proq kontekstni tushunish va hisoblash xarajatini kamaytirish.

4 Tadqiqot tuzoqi

Katta model (100B+) har doim ham 100% yaxshiroq emas. Ko'pincha 100B model atigi 5% ga yaxshi natija beradi, lekin hisoblash resursi 2% ga ko'proq bo'ladi. Shuning uchun har doim model hajmi va resurs o'rtasidagi muvozanatni tanlash muhim.

5 Qisqacha tushuncha: Attention Layer



Bu jarayon modelga "qaysi so'zlarga ko'proq e'tibor berish kerak?" degan savolga javob beradi.

6 Qisqacha tushuncha: MLP Layer



MLP qatlamida har bir token alohida qayta ishlanadi. ReLU kabi aktivatsiya funksiyasi modelga nolinearlik (egri chiziqli) imkoniyatini beradi.

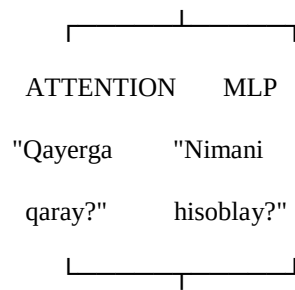
Asosiy xulosa

- Kichik model (100M)** — kamroq resurs, tezroq, lekin imkoniyati cheklangan.
- Katta model (100B+)** — ko'proq imkoniyat, lekin ko'proq hisoblash resursi talab qiladi.

- Attention va MLP qatlamlari** Transformer'ning asosiy bloklari hisoblanadi.
- Attention optimizatsiyasi** modelning samaradorligini oshiradi.
- Model hajmini tanlashda** "eng yaxshi natija + resurslar" muvozanati muhim.

SLM da attention va MLP layers teng lekin LLM da MLP layers roli katta Attention esa ahamiyatsiz

TRANSFORMER



Yaxshi representation

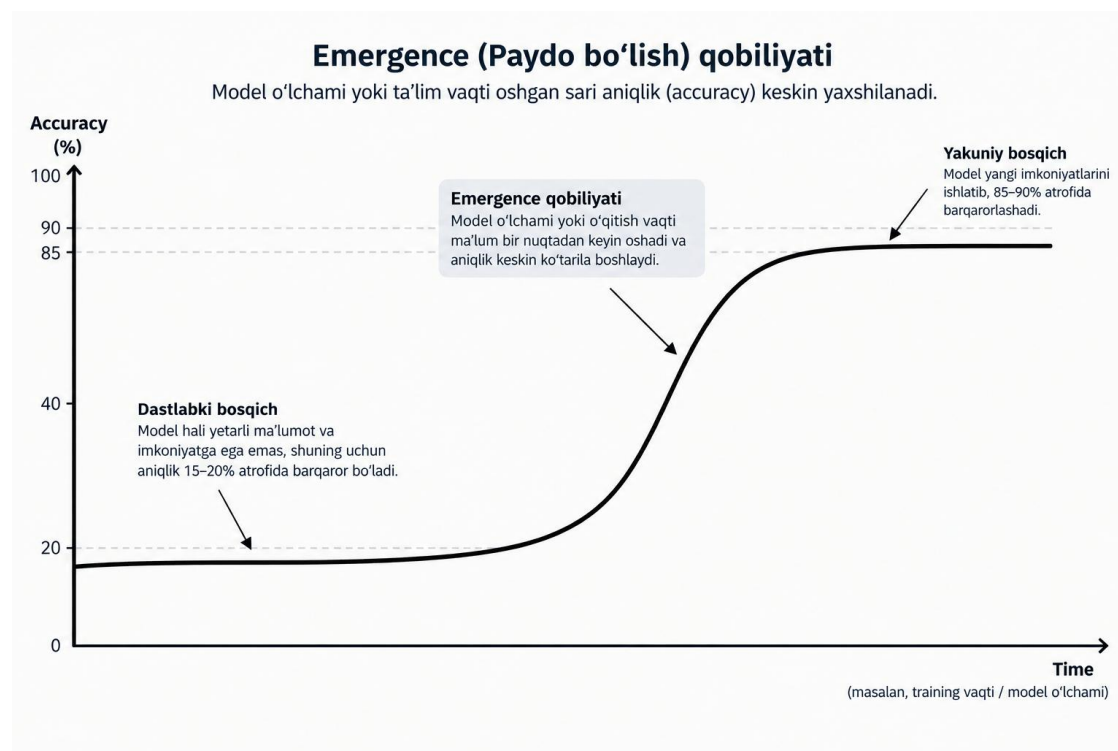
Attention → kontekst / tokenlar orasidagi aloqa

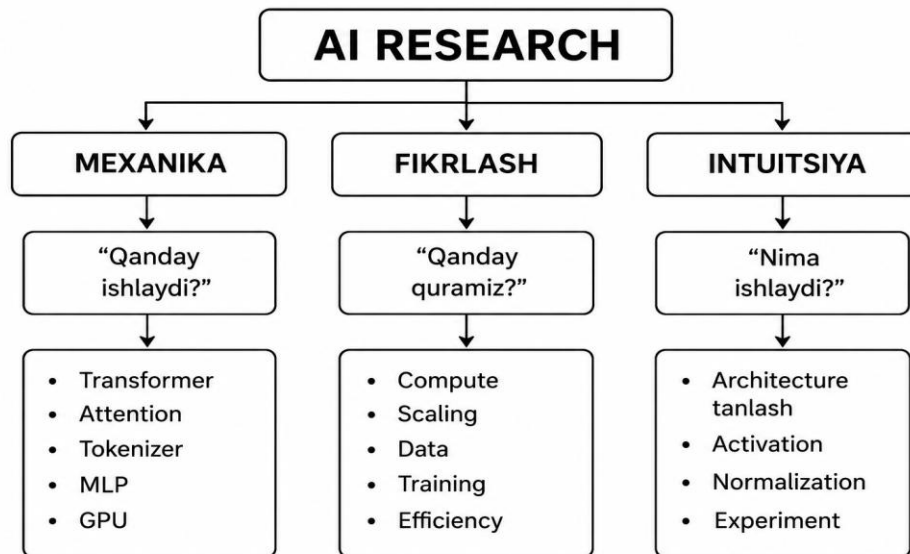
MLP → ma'lumotni chuqur qayta ishlash / pattern va representation

"The cat sat on the mat because it was tired." → ATTENTION LAYER → "it" → "cat" bilan bog'liq →

→ MLP shu ma'lumotni qayta ishlaydi, yangi representation qiladi

EMERGENCE - bu kutilmagan qobiliyat. Mana bu rasmda ko'rinib turganidek dastlab malum vaqt stabil bo'lib birdaniga oshib ketishi bu emergence qoibilyati





Tuqoridagi 3 ta asosiy bilimni anglab tushunish muhim yani misol uchun Transformer nima bu LLM yuragi yani LLM ning Neyrol networki.

The Bitter Lesson - AI yaratishda faqat qo'lda berilgan bilimlarga tayanmasdan, umumiy algoritmlar + katta compute + samaradorlik + yaxshi architecture birgalikda ishlatilishi kerak deydi. Copute katta bo'lsa g'ala qiladi bu xatô.

Optimizatsiya muammosi - cheklangan compute budget bilan yaxshi model qurishdagi muhim resurslar:

1. Compute- qancha parameter, qancha token va qanday batch size ishlatilishini to'g'ri tanlash kerak.
2. Data – data miqdori emas, uning sifati ham muhim. Datasetda keraksiz yoki takroriy datalar bo'lsa filter qilish muhim.
3. memory (VRAM)- train jarayonida ma'lumotlar GPU da saqlanadi va yetarli xotira bo'lishi kerak 1marta train uchun
4. hardware texnikalar interconnect ini tushunishi kerak.
5. inson kuchi

Compute + data + memory + hardware + insoniy bilim hammasi qo'shilib natija yaxshi model

Attention mexanizm vs Transformer architecture

Attention mexanizmi har bir tokenning boshqa tokenlar bilan bog'liqligini hisoblaydi va muhim so'zlarga ko'proq e'tibor beradi. MLP layer esa Attentiondan olingan ma'lumotni qayta ishlab, token tasvirini o'zgartiradi va modelga bilimni saqlashga yordam beradi. Tokenizer matnni tokenlarga, keyin esa integer ID larga aylantirib, modelga input sifatida beradi. qisqacha Attention kontekstni tushunadi, MLP esa ma'lumotni qayta ishlaydi.

Scaling law

Scaling Law — model parameterlai, tokenlari va compute (FLOPs) oshirilganda model sifati qanday o'zgarishini ko'rsatadigan bog'liqlik. Bu qoida yordamida katta modelni training qilishdan oldin taxminiy natijani hisoblash mumkin.

$$C = 6 \times N \times D$$

- C — training uchun kerak bo'ladigan FLOPs
- N — parameterlar soni
- D — training tokenlari soni
- 6 — forward pass taxminan $2 \times N \times D$, backward pass esa $4 \times N \times D$ compute talab qilgani uchun: $2 + 4 = 6$.

Chinchilla Law — model hajmi va training ma'lumotlari o'rtasida qanday nisbat bo'lishi kerakligini tushuntiradigan qoida va model shunchaki kattalashtirish yetarli emasligini tushuntiradi. Tokenlar parametrlardan 20 baravar ko'p bo'lishi kerak. Oldin ko'plab katta modellar parameterlari ko'p bo'lsa ham, yetarli token bilan o'qitilmagan. Chinchilla tadqiqotidan keyin kattaroq modelni kamroq token bilan o'qitishdan ko'ra, kichikroq modelni ko'proq token bilan o'qitish samaraliroq bo'lishi mumkinligini ko'rsatdi. Token kop bo'lsa data ham katta.

Quantity, Quality

AI modelini o'qitishda faqat data miqdori emas, balki data sifati ham juda muhim. **raw data** (xom, tekshirilmagan ma'lumotlar) modelga to'g'ridan-to'g'ri berilmasligi kerak. **quality filtering** orqali keraksiz, sifatsiz yoki noto'g'ri ma'lumotlar filter qilinadi. Keyin **deduplication** orqali bir xil yoki takrorlangan ma'lumotlar olib tashlanadi. **Language filtering** esa kerakli tillardagi ma'lumotlarni tanlaydi agar model faqat ingliz tiliga mo'ljallangan bo'lsa, ingliz tilidagi ma'lumotlarga e'tibor beriladi **toxicity/quality** filteri orqali zararli, haqoratomuz so'zlar olib tashlanadi.

SFT (Supervised Fine-Tuning) oldindan o'qitilgan modelni **prompt va ideal javob juftliklari** orqali qayta o'qitishdir. Misol uchun: **Prompt:** “5 yoshli bolaga suv aylanishini tushuntir.”

Ideal javob: “Quyosh suvni isitadi, suv bug'lanadi...”

Alignment - maqsadi modelning xatti-harakatlarini insonga **foydali, xavfsiz va kerakli yo'nalishga** moslashtirishdir. SFT alignment jarayonining muhim qismlaridan biri bo'lishi mumkin.

RLHF (Reinforcement Learning from Human Feedback) — LLMni insonlarning fikr-mulohazalari asosida yaxshilash bir xil savolga berilgan ddeylik turli javoblari ko'rsatiladi va ular qaysi javob yaxshiroq ekanini inson tanlaydi keyin shudan **reward oladi**. Reward olganini ko'proq keyin javob qilib beradi. keyin **RL fine-tuning** orqali LLM yuqori reward beradigan, ya'ni insonlar afzal ko'rgan javoblarni ko'proq berishga o'rgatiladi. Pre-training — bilim beradi, SFT — javob berishni o'rgatadi, RLHF esa modelni **inson afzalliklariga moslashtiradi**.

SFT vs RLHF

SFT (Supervised Fine-Tuning) — modelni inson yozgan **ideal javoblar** orqali o'qitish. **RLHF (Reinforcement Learning from Human Feedback)** — model javoblarini insonlar **taqqoslab tanlaydi**, keyin bu tanlovlardan reward modeli yaratiladi. LLM reward asosida yaxshi deb baholangan javoblarni ko'proq berishga o'rgatiladi. SFTda model **“to'g'ri javob qanday?”** degan namunadan o'rganadi. RLHFda esa **“qaysi javob insonlarga ko'proq yoqdi?”** degan feedbackdan o'rganadi.

Tokenization — raw matnni model qayta ishlay oladigan kichik birliklarga, ya'ni **tokenlarga** ajratish jarayoni. Tokenlar keyinchalik sonli ID larga aylantiriladi va modelga input sifatida beriladi. Tokenization usuli modelning vocabulary hajmi, sequence uzunligi va hisoblash xarajatiga ta'sir qiladi. Asosiy tokenization usullari **Character, Byte, Word va BPE** hisoblanadi. **Character tokenization** matnni har bir belgiga ajratadi. Vocabularyda mavjud bo'lmagan yangi so'zlarni ham qayta ishlay oladi, lekin sequence juda uzun bo'ladi. **Byte tokenization** har bir byte bilan ishlaydi va deyarli barcha matnni qabul qila oladi, ammo sequence yanada uzun bo'lishi mumkin. **Word tokenization** matnni to'liq so'zlarga ajratadi. Bu usulda sequence qisqaroq bo'ladi, lekin yangi yoki vocabularyda mavjud bo'lmagan so'zlar unknown sifatida chiqishi mumkin. **BPE (Byte Pair Encoding)** esa so'zlarni to'liq saqlash yoki har bir belgiga ajratish o'rtasidagi usuldir. U ko'p uchraydigan belgilar yoki token juftliklarini

birlashtirib, **subword** tokenlar yaratadi. Masalan, yangi yoki uzun soʻz bir nechta tanish qismlarga boʻlinishi mumkin. BPE ning muhim afzalligi yangi soʻzlarni ham qayta ishlay olishi, vocabulary hajmini nazorat qilish va sequence uzunligini **character, byte** usullariga qaraganda kamaytirishidir. Shu sababli BPE va unga oʻxshash subword usullari zamonaviy LLMlarda keng qoʻllanadi. Tokenization modelga matnni tushunarli sonli birliklarga aylantirib beradi

Atamalar jadvali:

	Term	Defination
1	LM LLM	LM - (predict next word) bu konteksga asoslangan xolda ketma ket keyin keladigan soʻz yoki gapni predict qiladigan model LLM - bu esa LM ning advanced darajasi boʻlib juda katta dataset bilan milliard yoki tirrillionlab parameter tokens bilan train qilingan model
2	Frontier models	Frontier model bu eng zamnaviy, eng oxirgi, eng kuchli va ulkan compute clusterlarda train qilingan modellar . Misol uchun (claude)
3	Emergence	Emergence- model datslab train jarayonida maʼlum vaqt davomida past stabil boʻlib turadi va keyin birdan yaxshilanib ketadi yani sharp increase boʻladi. Bu emergence holati kichik modellarda yuz bermaydi
4	FLOP	FLOP (Floating Point Operations) — kompyuter modelni hisoblash paytida bajarayotgan bitta matematik amal. Misol uchun 2.3×4.7 bu bitta FLOP
5	Scaling law	Scaling law qoidasi bu model parameterlari, tokenlari va compute (FLOPs) oshirilganda model sifati qanday oʻzgarishini koʻrsatadigan bogʻliqlik. Bu qoida yordamida katta modelni training qilishdan oldin taxminiy natijani hisoblash mumkin.
6	Bitter lesson	AI yaratishda faqat qoʻlda berilgan bilimlarga tayanmasdan, umumiy algoritmlar + katta compute + samaradorlik + yaxshi architecture birgalikda ishlatilishi kerak deydi. Compute katta boʻlsa gʻala qiladi bu xato
7	Alignment	Alignment - maqsadi modelning xatti-harakatlarini insonga foydali, xavfsiz va kerakli yoʻnalishga moslashtirishdir. Alignment boʻlmasa model insonga zarar beradigan bilimlarni ham beradi ex: Nuclear weapons
8	SFT	SFT (Supervised Fine-Tuning) oldindan oʻqitilgan modelni prompt va ideal javob juftliklari orqali qayta oʻqitishdir. Ex: Odam birona ideal javob yozsa shu orqali oʻqitish
9	RLHF	Bunda model javoblarini insonlar taqqoslab tanlaydi yani qanday 2 ta javob beriladi qay biri foydalanuvchiga maqul boʻlsa keyin yaxshi deb baholangan javoblarni koʻproq berishga oʻrgatiladi. SFTda model “toʻgʻri javob qanday?” degan namunadan oʻrganadi. RLHFda esa “qaysi javob insonlarga koʻproq yoqdi?” degan feedbackdan oʻrganadi
10	Tokenization	Tokenization - raw matnni model qayta ishlay oladigan kichik birliklarga, yaʼni tokenlarga ajratish jarayoni.
11	Chincilla law	Bu model hajmi va training maʼlumotlari oʻrtasida qanday nisbat boʻlishi kerakligini tushuntiradigan qoida va model shunchaki kattalashtirish yetarli emasligini tushuntiradi. Tokenlar parametrlardan 20 baravar koʻp boʻlishi kerak. Ex: 1B parameter 20B tokens

